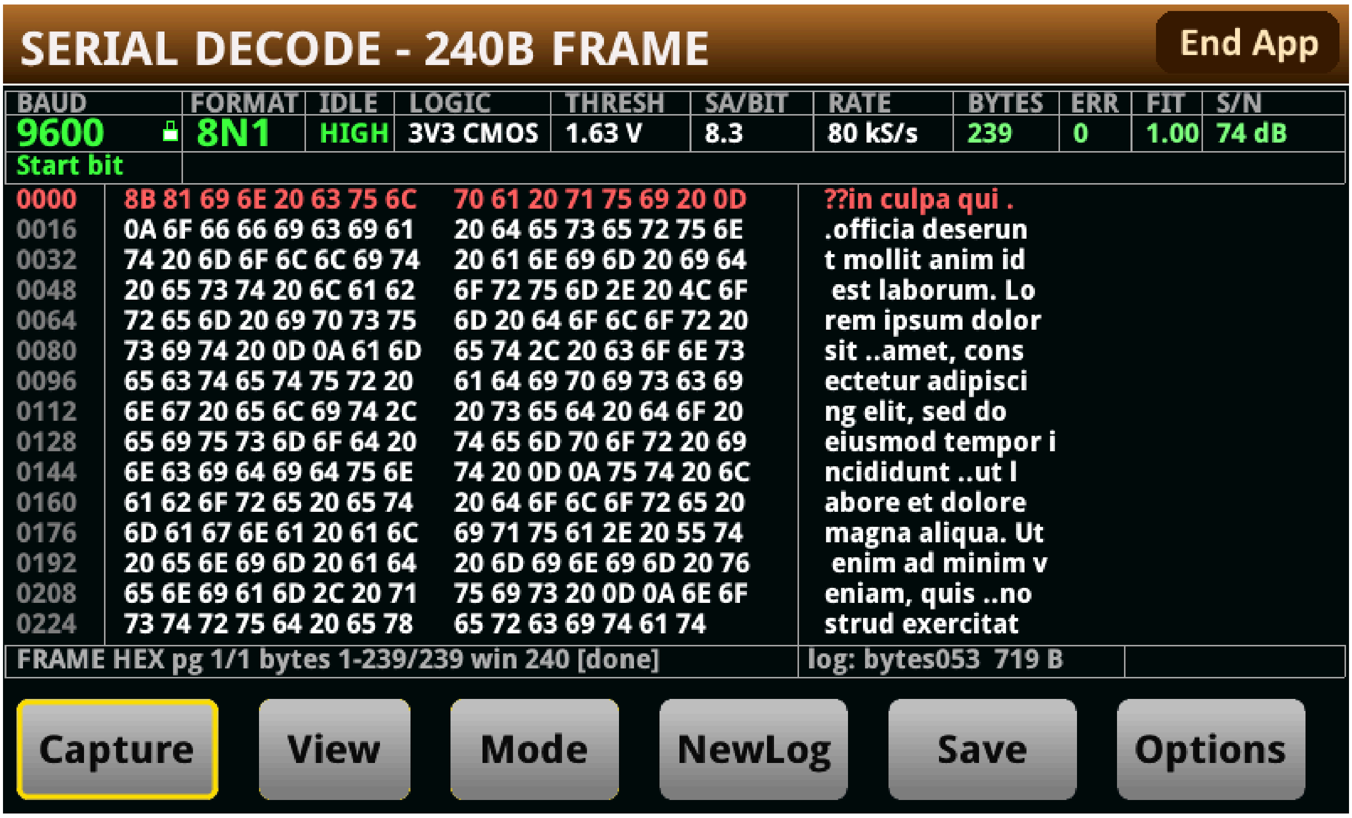
Serial Decode — Keithley DMM6500, DAQ6510, DMM7510, SMU2461

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.20** · MIT licence (see LICENSE)



The instrument's own front panel, no host attached: 239 bytes of a 240-byte capture at 9600 baud 8N1, no framing errors, S/N 74 dB. The padlock on the BAUD cell means the rate is pinned here rather than re-detected each capture.

docs/MANUAL.md	using it: hooking up, the screen, the buttons, what it copes with, where it fails
docs/REFERENCE.md	every measured number, and the firmware limits behind them
docs/BENCH.md	the harness: the seeded sweep, the release gate stage by stage, reproducing a failure

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the firmware, so the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

What to know before trusting it

Tested on a DMM6500 and nothing else. Every measured number in these documents came off that one instrument. See Which instruments.

Flow control is verified electrically but never closed as a loop. One credit pulse per armed capture, measured on a scope, with nothing waiting for it on the other end. The width is not settable on this firmware, so a receiver needs an edge-triggered input rather than a polling loop. See **Triggering, in both directions** in REFERENCE.md.

Automatic rate detection has three known failures — **8 points in 860** on a full bench plan for the first two and about 3 in a million for the third — and all three are fixed by typing the rate in: a short pattern repeated over and over can measure at a small multiple of its true rate, a logic low near **–2 V** can misdetect, and above 115 200 baud a long run of one repeated byte can refuse. **None reaches an ordinary payload at a standard baud rate:** across 171 600 captures at standard rates, opened from 200 different points in the waveform, the fox, the 1 kB text payload, twelve random payloads and the walking-bit patterns are clean at every rate from 300 to 250 000 baud. All three are characterised under **Known failures of automatic rate detection** in the manual.

Nothing stops a long job early. A touch press cannot be delivered while a script runs, and the front-panel TRIGGER key does **not** deliver `trigger.EVENT_DISPLAY` during a panel-initiated run, so a recording runs to its stated size. That key does still *gate* an armed capture under `Options ▸ Trigger = Trigger key`.

One press is the whole interaction. A screenful is ~240 bytes; a recording holds 8 192 or 32 768 bytes to a file, decoded and filed without stepping; beyond that, flow control makes the window a chunk size and credits the device until it stops sending. Ceilings and arithmetic are in the manual.

Endurance, measured

Seven and a half days with no computer attached, 136 247 captures, not one instrument event. 182.2 h, 08-29 to 09-05, twelve segments of the instrument's own bench loop: it read its plan from its USB key, drove the generator over the LAN itself, and wrote every cell back to the key. Every segment's closing total reports **0 events instigated, 0 from the panel** — on this firmware an event is a modal box nothing suppresses — and **0 cells unrecorded in 0 gaps**. The generator needed **270 waveform re-selections**, all recovered: 241 on the second try, 28 on the third, 1 on the fourth.

	duration	cells	judged BAD	rate
the week, 08-29 → 09-05	182.2 h	133 297 judged	2 462	**1.85 %**
the night of 09-07	17.24 h	11 739	143	**1.22 %**

The second is the only long run on the current build — 7 laps of the 1 677-point matrix on one power cycle. **The two rates are not comparable:** the week ran an older build *and* a different matrix, laps of 1 326 and 1 333 cells with a different vector set, amplitudes and waits. The drop is not a measured improvement.

A low single-digit rate is the expected shape, because the vectors are deliberately hostile: impulse spikes stacking to 9.3 V on a 3.3 V line, noise and a second signal riding on the logic from generator CH2 summed into CH1 at a sweepable level, baud drifting 6 % and 10 % mid-waveform, 2/10/20 % jitter, an inverted line, LIN break fields no UART frame can legally contain, 256- and 512-byte runs of one value with almost no transitions, payloads longer than the capture — and **every one swept across all 43 rates**, including rates two decades off the one it was built for.

Composition is what stops that being an excuse. Only the seven vectors built to break something may fail — `v47`, `v48a/b`, `j20`, `v61/62/63`, class `loud` in `tools/soakplan.py` — and they carry just **32 % of the week's failures; the other 68 % are `exact` vectors, where a failure is a defect** and counted as one. **A confidently wrong byte fails every class**, `loud` included: refusing is a pass, being wrong while claiming to be right never is.

The 17-hour run also **leaked nothing** — heap after collection at the six lap seams read 2678, 2678, 2679, 2678, 2679, 2678 and 2678 kB, flat to 1 kB over 10 062 cells, and lap time did not rise, which catches a leak no failure count can. **`line is idle` did not occur once**, which rules out instrument uptime by itself, and **generator failures were 0** against 129 in the run before it — all of those one mis-specified skip list rather than the instrument.

Confidently wrong bytes, in any of it: none. Every failure is the app reporting a rate its own framing contradicts, or declining. Separately, over **1 066 400 offline decodes: 95.9 % of failures are the rate being wrong and 0.026 % get a byte wrong with the rate right** — see **Rate detection and decode, counted separately** in REFERENCE.md. The offline twin is deliberately pessimistic and checked against these runs rather than trusted: over **167 700 cells in 100 laps** it fails on 1.785 % where the bench fails on 1.210 %, **over-predicting by 1.48×** — and which way that error points is the property worth having (BENCH.md).

Which instruments

The app needs a digitizer reachable as **`dmm.digitize`** or **`smu.digitize`**, the **touchscreen app API** (`display.create` and friends), and **Lua 5.0.2**. Every digitizer below runs at **1 MS/s**, and the **panel is pixel-identical across the whole TSP range**, so the app's screens carry over as they are.

DMM6500	**Tested**, firmware 1.7.17a — a 7.6-day run of 136 247 captures with no computer attached and zero instrument events, a 17.24-hour run on the current build, and the namespace resolver verified here. 16-bit digitizer, "*maximum resolution 16 bits*", specifications, April 2018
DAQ6510	Should run unmodified, on the strongest grounds of any untested model: it **shares the UI board and the acquisition board** with the DMM6500, only the channel-board plugin differing. Untested
DMM7510	Should run unmodified: 18-bit digitizer, better acquisition boards. Untested
SMU2461	**Will install and try; may or may not work.** Dual 18-bit digitizers, reached as `smu.digitize` by a namespace the app resolves at load. That mechanism is verified on the DMM6500; no SMU has ever run it. Three unknowns below
2470 SMU	**No.** "*Digitized measurements are not a feature on the 2470*" — its reference manual, rev D, October 2024
2450 / 2460 SMU	Almost certainly not, by absence rather than denial: neither claims a digitizer, and the 2450 reference (rev D, May 2015) documents `smu.measure.*` with no digitize function

Porting is an acquisition question only. The panel is identical, so the display layer carries over untouched, and the app resolves `dmm.digitize` or `smu.digitize` once at load with no test on any capture path. Every rate figure in REFERENCE descends from the sample rate and the reading buffer's throughput, so the timing figures transfer; resolution is the one axis the app is indifferent to, thresholding each sample into a one or a zero. What to re-measure is what goes *through the acquisition board* — the tolerance envelope, the level and offset limits, the –2 V band. The DAQ6510 shares that board, so a deviation there is a real finding; the DMM7510's differs and those figures may move either way.

On an SMU2461 the app will install and try, and three things could stop it. *What the reading buffer holds* — a 2461 digitizes voltage and current **simultaneously** and the decoder trusts consecutive indices, so a buffer carrying current or source values alongside voltage reads as a waveform and surfaces as garbage bytes rather than an error. *The range* — the app pins **10 V**, chosen for the widest digitize bandwidth, which a 2461 may not offer. *The source output* — the app sets it off before every capture and **reads it back, refusing the capture if it cannot confirm it**; those constant names are unverified, and that refusal is the failure it is designed for.

The `.tspa` header decides where it installs, independently of whether the code would run: `$Product: DMM6500, DAQ6510, DMM7510, SMU2461`. Neither `DMM7510` nor `SMU2461` appears in the value set Keithley's app-header spec publishes, though Keithley ship TSP apps of their own declaring DMM7510 — an install failure is the symptom if a firmware validates the field strictly, and this field is the first thing to revert.

If you run this on anything but a DMM6500, please say so — open an issue. Every row above except the first is inference from API surfaces and vendor documents, and one report from real hardware outweighs all of it. Useful to include: `localnode.model` and `localnode.version`, whether **Manage Apps** offered the app at all, whether both screens built, and whether a capture decoded.

How to report bugs

docs/BETA.md (PDF) is the note to read first, and the one to hand to anybody trying this on their own bench. It lists the four behaviours that **look** like defects and are not — chiefly that a signal whose band straddles ground is decoded inverted, deliberately — so a tester does not spend their evening rediscovering them.

Then open an issue with, as far as you can: a **screen grab** of the whole 800x480 frame (`tools/screenshot.py` takes one over LXI with nothing loaded), the **text that should have been decoded**, the **bit rate** you set, and the **framing** you sent. The expected payload is the one piece that cannot be reconstructed from anything else — it turns "the decode looks wrong" into a difference that can be computed.

A wrong number that looks plausible is worth more than a crash.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order.

```
python3 tools/release_sweep.py --offline   #  ~1 min, no instruments
python3 tools/bench_smoke.py               #  13 min, both instruments
python3 tools/soak.py --hours 17 --suites formats,plan --skip-vectors v95,v96
python3 tools/release_sweep.py             #  the whole thing, instruments included
```

The smoke gate covers the whole rate range in 13 minutes. Four waveforms — the fox and a random payload, each in 8N1 and 7E1 — paired crosswise, so one pair takes the 22 standard baud rates and the other the 21 drawn ones, and each rate family faces both formats and both content classes. Then all 45 button presses, seven logic swings from 5 V down to 0.25 V, and eight wrong-locked-rate cases. Measured: 86 cells in 9.7 min, 45 presses in 1.9, 7 levels in 0.7, 8 rate cases in 0.9.

No version is tagged without at least 8 hours of soak. A soak reports a **failure rate per test point**, where a sweep reports one pass or one failure. A lap is 1 683 cells and about 2.9 hours, so 8 hours is the least that separates "fails every lap" from "failed once", and 17 gives six laps.

Code changes do not get pushed without passing the smoke gate. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/`; a `pre-push` hook recomputes it and refuses if either tree has moved since. Documentation-only pushes are unaffected — neither tree is hashed — and when the bench is unavailable, `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason.

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Every lap draws its waveform order, its non-standard rates and the wait before each capture from the lap number, so `--iteration N` rebuilds any lap exactly without running the ones before it, and a failure replays offline in seconds.

docs/BENCH.md has the rest: every stage of the release gate and what it checks, the auditable record a full sweep leaves behind, what state the instruments have to be in before it will start, how to replay a failing cell offline, and the hazards worth knowing.

Layout

`tsp/`	the app. `serial_core` acquisition, `uart_decode` framing, `chunk_decode` resumable decode, `serial_ui` panel, `serial_app` orchestration
`bench/`	the soak that runs **on** the instrument, loaded beside the app for a multi-day run and never shipped inside it. bench/README.md is the runbook
`tools/`	harnesses. `release_sweep.py` is the entry point; the rest are the authorities it calls
`docs/`	the manual, instrument references, panel mockups, and BETA.md for testers

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Two instruments over LAN: the DMM6500 under test and an SDG2000X-series generator producing the stimulus. **A scope is optional** — an SDS1000X-E is used here to confirm the stimulus is what it claims to be, and nothing in the gate needs it; it only ever reads. `tools/instruments.py` holds the addresses and the hazards: repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.

Neither line's top model is needed. The bench asks the generator for 25 MSa/s of TrueArb, 20 Vpp and ~41 stored waveforms, and the scope, if used, for UART decode, two buses and 1 GSa/s; in both lines the model number is the *analog bandwidth*, and the fastest edge here is a 250 kBd bit. An **SDG2042X (40 MHz) and an SDS1104X-E (100 MHz) are sufficient**; the units here are an SDG2122X and an SDS1204X-E. `SDG_MAX_SRATE` is capped at 40 MSa/s so a plan cannot quietly outgrow the cheapest generator; see docs/BENCH.md for what to check on a unit that is not the one on this bench.